



국립 한국해양대학교  
NATIONAL  
KOREA MARITIME & OCEAN UNIVERSITY

# 친환경스마트조선기자재학과

## 데이터처리특론



# 파이썬(Python)



## 파이썬

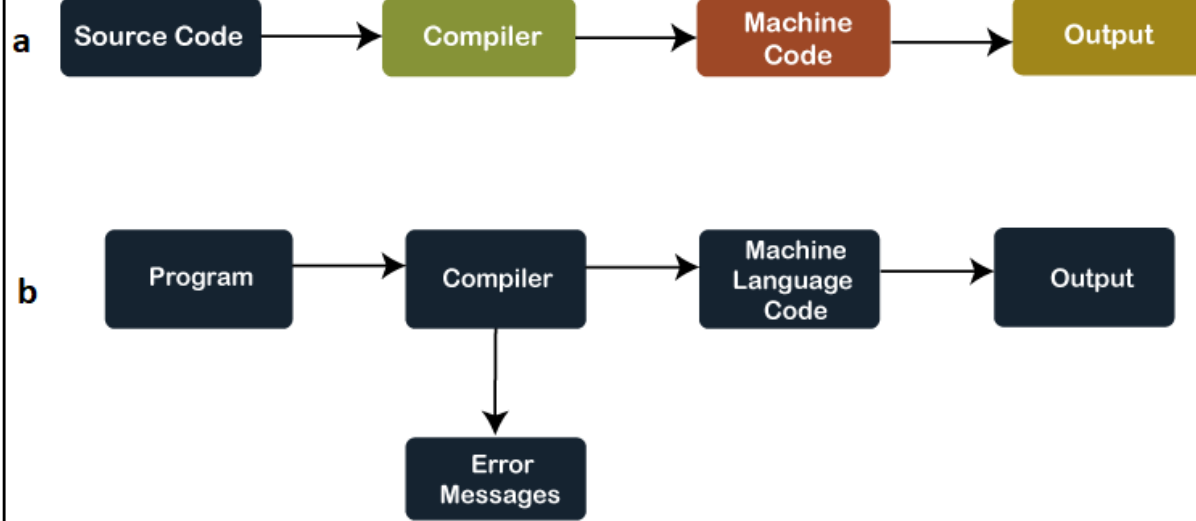
- 파이썬은 1991년에 귀도 반 로섬(Guido van Rossum)에 의해 만들어진 인터프리터 프로그래밍 언어
- 직관적이고 쉬우며 특징과 다양하고 풍부한 라이브러리들을 바탕으로 한 강력한 생태계를 가지고 있음
- 프로그래밍 교육, 인공지능, 데이터 분석 및 빅데이터, 백엔드, 프론트엔드, 웹 스크래핑 등 다양한 분야에서 사용
- AI와 빅데이터의 부상과 함께 최근 각광받고 있는 언어

| Feb 2024 |  | Feb 2023 |  | Change |  | Programming Language                                                                  |            | Ratings |  | Change |  |
|----------|--|----------|--|--------|--|---------------------------------------------------------------------------------------|------------|---------|--|--------|--|
| 1        |  | 1        |  |        |  |   | Python     | 15.16%  |  | -0.32% |  |
| 2        |  | 2        |  |        |  |  | C          | 10.97%  |  | -4.41% |  |
| 3        |  | 3        |  |        |  |  | C++        | 10.53%  |  | -3.40% |  |
| 4        |  | 4        |  |        |  |  | Java       | 8.88%   |  | -4.33% |  |
| 5        |  | 5        |  |        |  |  | C#         | 7.53%   |  | +1.15% |  |
| 6        |  | 7        |  | ▲      |  |  | JavaScript | 3.17%   |  | +0.64% |  |

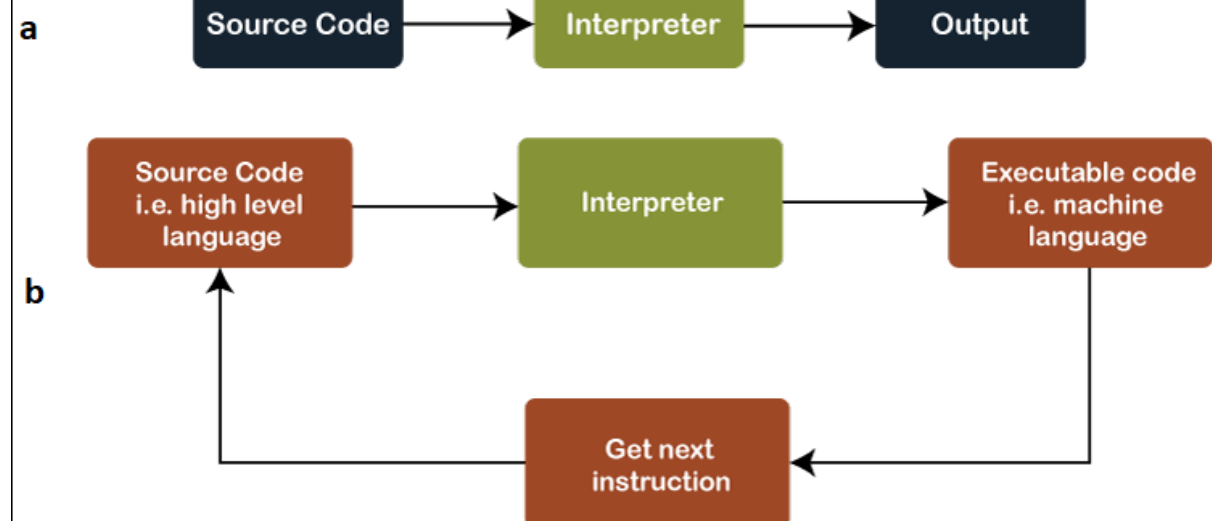


# Compiler vs Interpreter

## How Compiler Works



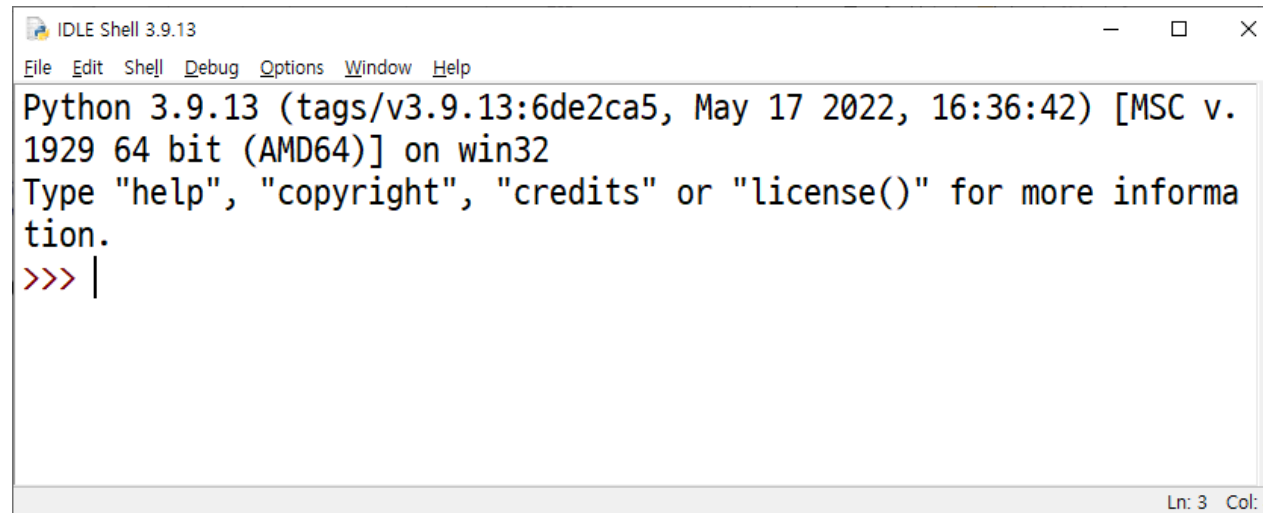
## How Interpreter Works



## 파이썬 실행하기

- CLI 환경에서 실행하기
  - > python
- IDLE 실행하기
  - 시작 – 파이썬 - IDLE

```
Type "help", "copyright", "credits" or "license" for more information.  
>>>
```



## 통합개발환경

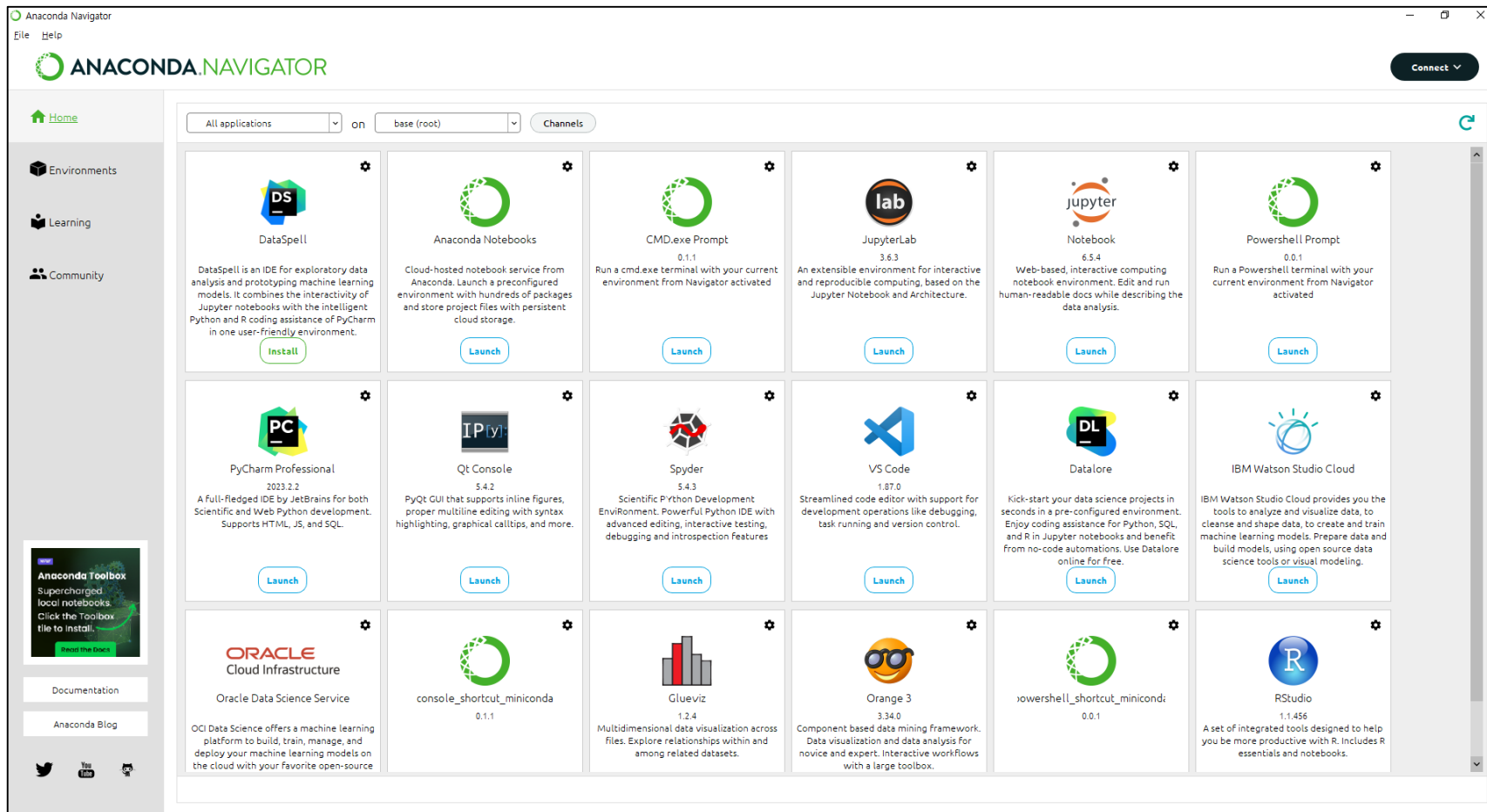
- Jupyter
  - 통합개발환경이라기 보다 웹에서 Python 코드 작성 및 Markdown 문서 작성이 가능한 웹어플리케이션
  - 웹 상에서 Python 코드를 작성하고 그 결과를 바로 확인 가능함.
  - Anaconda 배포본에 포함되어 있음
- Spyder
  - Anaconda 배포본 사용시 제공되는 IDE로 과학기술계산용으로 특화
  - Crossplatform이며 무료
  - 간단하게 사용할 수 있는 IDE

## 통합개발환경

- Visual Studio PTVS
  - Visual Studio에 PTVS라는 확장을 인스톨하면 Python용 IDE로 사용가능.
  - Windows 용이며, 개인개발자는 무료이며 그외는 상용
  - 최고의 Windows용 IDE
- PyCharm
  - 가장 많은 사용자가 있는 Python IDE
  - Crossplatform이며 무료 및 유료 버전 있음.
  - 다양한 확장기능을 설치할 수 있음
  - 가장 많은 사람의 선택을 받는 IDE
- Visual Studio Code
  - Visual Studio Code에 Python 확장을 인스톨하면 Python용 IDE로 사용가능.
  - Crossplatform이며 무료
  - 다양한 확장기능을 설치할 수 있음
  - 최근 사용자 층이 늘어나고 있는 IDE

## 아나콘다(Anaconda)

- 파이썬과 R 개발자를 위해 여러 도구를 모아둔 집합체, 오픈 소스 배포판



# Keyword

## 키워드(keyword)

- 각 프로그램 언어에서 특정 용도로 사용하기 위해 만들어진 특별한 의미를 가지는 예약된 단어

```
>>> import keyword  
>>> keyword.kwlist
```

```
import keyword  
# print(keyword.kwlist)  
kwlist = keyword.kwlist  
for i in range(0,len(kwlist)):  
    print("{:10}".format(kwlist[i]), end=" ")  
    if (i+1)%5==0: print()
```

```
[False      ] [None       ] [True       ] [and        ] [as         ]  
[assert     ] [async      ] [await      ] [break      ] [class      ]  
[continue   ] [def         ] [del        ] [elif       ] [else       ]  
[except     ] [finally    ] [for        ] [from       ] [global     ]  
[if         ] [import     ] [in         ] [is         ] [lambda     ]  
[nonlocal   ] [not        ] [or         ] [pass       ] [raise      ]  
[return     ] [try        ] [while      ] [with       ] [yield      ]
```



# 리터럴과 자료형

| 분류  | 내장 자료형             | 리터럴                                          |  |  |
|-----|--------------------|----------------------------------------------|--|--|
| 수치  | 정수(int)            | 10      -30      0xfffe      073             |  |  |
|     | 실수(float)          | 3.14      -0.45      123.032E-13             |  |  |
|     | 복소수(complex)       | complex(1,2)      1+2j      4+5j             |  |  |
|     | 부울(bool)           | True      False                              |  |  |
| 시퀀스 | 문자열(str)           | 'game'      "over"      "C"                  |  |  |
|     | 리스트(list)          | []      [0, 1, 2, 3]      [0, 'hello', 3.14] |  |  |
|     | 튜플(tuple)          | (0, 1, 2, 3)      ('hello', 'world', 'game') |  |  |
| 매핑  | 딕셔너리(dict)         | { 3.14 : "phi", 4.5 : "score" }              |  |  |
| 집합  | 집합(set, frozenset) | { 1, 2, 3 }      {'one', 'two', 'three' }    |  |  |

# Variable, Constant, Literal

## 변수(variable)

- 하나의 값을 저장할 수 있는 저장공간

```
PI = 3.14  
GRAVITY = 9.8
```

## 상수(constant)

- 단 한 번만 값을 저장할 수 있는 저장공간
- 파이썬에서는 상수를 사용하지 않음

```
import constant1  
  
print(constant1.PI)  
print(constant1.GRAVITY)  
constant1.PI = 3.141592 # 변경된다. 상수가 아니다.  
print(constant1.PI)
```

## 리터럴(literal)

- 그 자체로, 존재 자체만으로 값을 의미하는 것

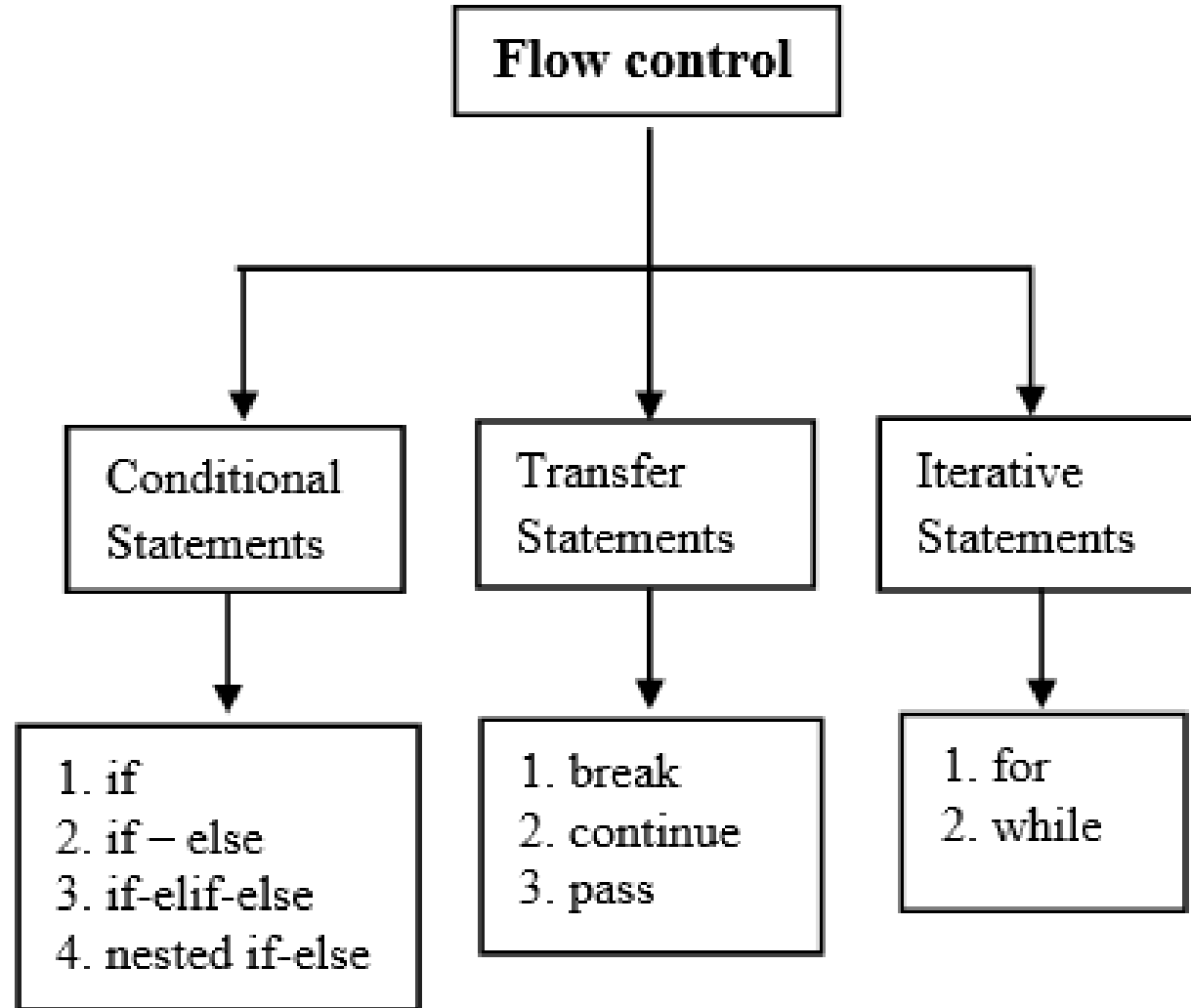
# Python operator priority

## 연산자 우선 순위(Operators Precedence)

| Precedence | Operator    | Description                                       | Associativity |
|------------|-------------|---------------------------------------------------|---------------|
| 1          | **          | Exponentiation                                    | Right to left |
| 2          | +x, -x      | Positive, negative                                | Right to left |
| 3          | *, /, %, // | Multiplication, division, modulus, floor division | Left to right |
| 4          | +, -        | Addition, subtraction                             | Left to right |
| 5          | <<, >>      | Bitwise shift left, bitwise shift right           | Left to right |
| 6          | &           | Bitwise AND                                       | Left to right |
| 7          | ^           | Bitwise XOR                                       | Left to right |

|    |                                                       |                                    |               |
|----|-------------------------------------------------------|------------------------------------|---------------|
| 8  |                                                       | Bitwise OR                         | Left to right |
| 9  | <, <=, >, >=                                          | Comparison operators               | Left to right |
| 10 | ==, !=                                                | Equality operators                 | Left to right |
| 11 | not                                                   | Logical NOT                        | Right to left |
| 12 | and                                                   | Logical AND                        | Left to right |
| 13 | or                                                    | Logical OR                         | Left to right |
| 14 | if-else                                               | Ternary conditional                | Right to left |
| 15 | =, +=, -=, *=, /=, //=, %=, **=, <<=, >>=, &=, ^=,  = | Assignment and compound assignment | Right to left |

# Control statement



# Control statement

## Range

```
for n in range( 5 ) :           # n: 0, 1, 2, 3, 4
```

```
for n in range( 2, 10 ) :       # n: 2, 3, ..., 9
```

```
for n in range( 10, 3, -2 ) :   # n: 10, 8, 6, 4
```

```
for item in [12, 33, 52, 26, 99] :   # 리스트의 모든 항목에 대해 반복
    print( "값 =", item)             # 12, 33, 52, 26, 99 출력
```

```
for c in "Game Over !" :           # 문자열의 각 문자에 대해
    print( "값 =", c)
```

```
mySet = set([12, 33, 52, 26, 99])
for e in mySet :
    print( "값 =", e)
```

```
myDict = { 'A':12, 'B':33, 'C':52, 'D':26, 'E':99 }
for e in myDict :
    print( "키 =", e)
    print( "값 =", myDict[e])
```

```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(1, 11))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> list(range(0, 30, 5))
[0, 5, 10, 15, 20, 25]
>>> list(range(0, 10, 3))
[0, 3, 6, 9]
>>> list(range(0, -10, -1))
[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
>>> list(range(0))
[]
>>> list(range(1, 0))
[]
```

# Collection Data Types

## LISTS VS. TUPLES VS. SETS VS. DICTIONARIES

|                    | Lists                 | Tuples                | Sets                                               | Dictionaries                                          |
|--------------------|-----------------------|-----------------------|----------------------------------------------------|-------------------------------------------------------|
| Ordering           | Ordered               | Ordered               | Unordered                                          | Ordered<br><small>Unordered before Python 3.7</small> |
| Indexing           | Indexed               | Indexed               | Not Indexed                                        | Keyed                                                 |
| Mutability         | Mutable               | Immutable             | Mutable<br><small>Only Adding and Removing</small> | Mutable                                               |
| Duplicates Allowed | Yes                   | Yes                   | No                                                 | Yes<br><small>Only in values and not in keys</small>  |
| Types Allowed      | Mutable and Immutable | Mutable and Immutable | Only Immutable                                     | Only Immutable<br><small>In keys</small>              |

## 1. 리스트는 어떻게 만들고 사용할까?

## 2. 리스트의 인덱싱과 슬라이싱

1. 리스트의 인덱싱
2. 리스트의 슬라이싱

## 3. 리스트 연산하기

1. 리스트 더하기(+)
2. 리스트 반복하기(\*)
3. 리스트 길이 구하기

## 4. 리스트의 수정과 삭제

1. 리스트의 값 수정하기
2. del 함수를 사용해 리스트 요소 삭제하기

## 5. 리스트 관련 함수

1. 리스트에 요소 추가하기 - append
2. 리스트 정렬 - sort
3. 리스트 뒤집기 - reverse
4. 인덱스 반환 - index
5. 리스트에 요소 삽입 - insert
6. 리스트 요소 제거 - remove
7. 리스트 요소 끄집어 내기 - pop
8. 리스트에 포함된 요소 x의 개수 세기 - count
9. 리스트 확장 - extend

# Tuple

1. 튜플은 어떻게 만들까?
2. 튜플의 요소값을 지우거나 변경하려고 하면 어떻게 될까?
  1. 1. 튜플 요소값을 삭제하려 할 때
  2. 2. 튜플 요소값을 변경하려 할 때
3. 튜플 다루기
  1. 인덱싱하기
  2. 슬라이싱하기
  3. 튜플 더하기
  4. 튜플 곱하기
  5. 튜플 길이 구하기



# Set

1. 집합 자료형은 어떻게 만들까?
2. 집합 자료형의 특징
3. 교집합, 합집합, 차집합 구하기
  1. 교집합 구하기
  2. 합집합 구하기
  3. 차집합 구하기
4. 집합 자료형 관련 함수
  1. 값 1개 추가하기 - add
  2. 값 여러 개 추가하기 - update
  3. 특정 값 제거하기 - remove

# Dictionary

1. 딕셔너리란?
2. 딕셔너리는 어떻게 만들까?
3. 딕셔너리 쌍 추가, 삭제하기
  1. 딕셔너리 쌍 추가하기
  2. 딕셔너리 요소 삭제하기
4. 딕셔너리를 사용하는 방법
  1. 딕셔너리에서 Key를 사용해 Value 얻기
  2. 딕셔너리 만들 때 주의할 사항
5. 딕셔너리 관련 함수
  1. Key 리스트 만들기 - keys
  2. Value 리스트 만들기 - values
  3. Key, Value 쌍 얻기 - items
  4. Key: Value 쌍 모두 지우기 - clear
  5. Key로 Value 얻기 - get
  6. 해당 Key가 딕셔너리 안에 있는지 조사하기 - in

# String

## 1. 문자열은 어떻게 만들고 사용할까?

1. 큰따옴표로 양쪽 둘러싸기
2. 작은따옴표로 양쪽 둘러싸기
3. 큰따옴표 3개를 연속으로 써서 양쪽 둘러싸기
4. 작은따옴표 3개를 연속으로 써서 양쪽 둘러싸기

## 2. 문자열 안에 작은따옴표나 큰따옴표를 포함시키고 싶을 때

1. 문자열에 작은따옴표 포함하기
2. 문자열에 큰따옴표 포함하기
3. 역슬래시를 사용해서 작은따옴표와 큰따옴표를 문자열에 포함하기

## 3. 여러 줄인 문자열을 변수에 대입하고 싶을 때

1. 줄을 바꾸기 위한 이스케이프 코드 \n 삽입하기
2. 연속된 작은따옴표 3개 또는 큰따옴표 3개 사용하기

## 4. 문자열 연산하기

1. 문자열 더해서 연결하기
2. 문자열 곱하기
3. 문자열 곱하기를 응용하기

## 5. 문자열 길이 구하기

## 6. 문자열 인덱싱과 슬라이싱

1. 문자열 인덱싱
2. 문자열 인덱싱 활용하기
3. 문자열 슬라이싱
4. 문자열을 슬라이싱하는 방법
5. 슬라이싱으로 문자열 나누기

## 7. 문자열 포매팅이란?

## 8. 문자열 포매팅 따라 하기

1. 숫자 바로 대입
2. 문자열 바로 대입
3. 숫자 값을 나타내는 변수로 대입
4. 2개 이상의 값 넣기

## 9. 문자열 포맷 코드

# String

## 10. 포맷 코드와 숫자 함께 사용하기

1. 정렬과 공백
2. 소수점 표현하기

## 11. format 함수를 사용한 포매팅

1. 숫자 바로 대입하기
2. 문자열 바로 대입하기
3. 숫자 값을 가진 변수로 대입하기
4. 2개 이상의 값 넣기
5. 이름으로 넣기
6. 인덱스와 이름을 혼용해서 넣기
7. 왼쪽 정렬
8. 오른쪽 정렬
9. 가운데 정렬
10. 공백 채우기
11. 소수점 표현하기
12. { 또는 } 문자 표현하기

## 12. f 문자열 포매팅

## 13. 문자열 관련 함수들

1. 문자 개수 세기 - count
2. 위치 알려 주기 1 - find
3. 위치 알려 주기 2 - index
4. 문자열 삽입 - join
5. 소문자를 대문자로 바꾸기 - upper
6. 대문자를 소문자로 바꾸기 - lower
7. 왼쪽 공백 지우기 - lstrip
8. 오른쪽 공백 지우기 - rstrip
9. 양쪽 공백 지우기 - strip
10. 문자열 바꾸기 - replace
11. 문자열 나누기 - split

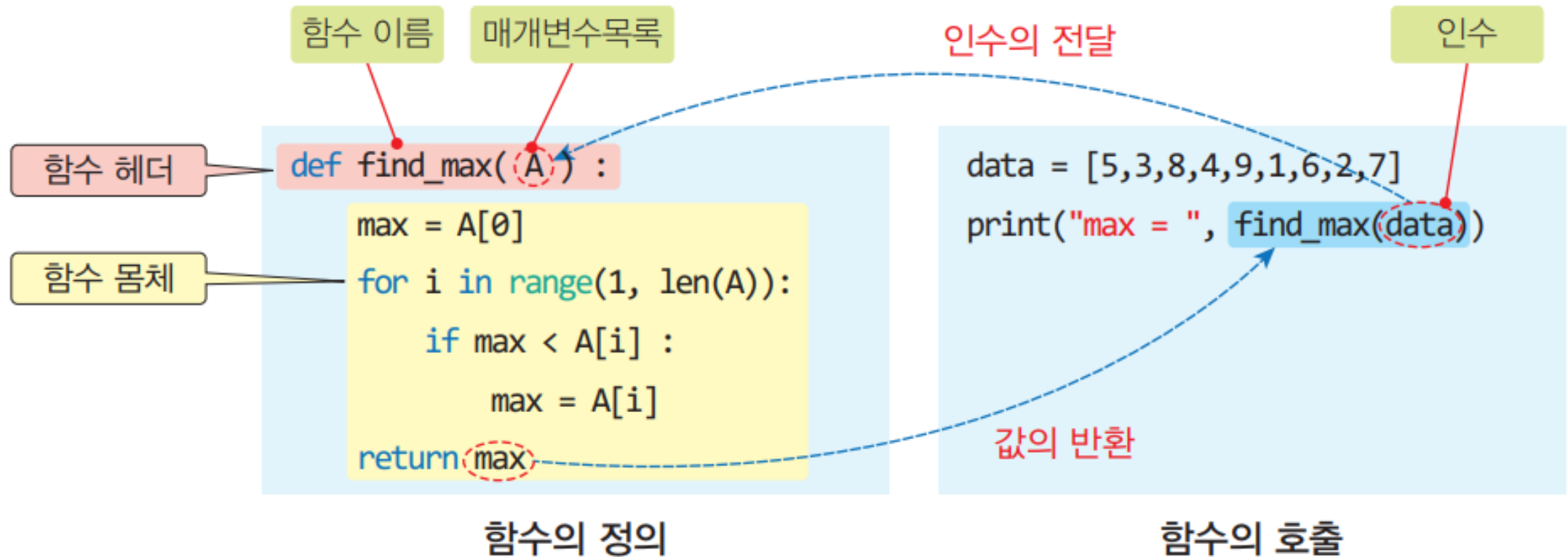
# Function & Module

| 구분 | 의미       | 예제                                                                                                                                                                                                                                        |
|----|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 함수 | 함수       | 특정 작업(task)을 수행하는 명령어의 모음                                                                                                                                                                                                                 |
|    | 내장함수     | Python 언어 자체에서 제공하는 함수<br><code>type(), int(), str(), list(), tuple(), chr(), ord(), range(), len(), input(), .....</code>                                                                                                                |
|    | 사용자정의 함수 | 사용자가 만들어 사용하는 함수<br><br><code>def user_fn(x, y) :<br/>    return x + y</code><br><br><code>sum = user_fn(10,6)</code>                                                                                                                     |
|    | 람다함수     | 함수 이름이 없는, 1회용으로 한줄에 정의하여 사용하는 함수<br><br><code>user_fn = lambda x , y : x + y</code><br><br><code>sum = user_fn(10,6)</code>                                                                                                              |
|    | 메소드      | 특정 객체(object)에 적용되는 함수<br><br><code>str.replace(), str.count(), str.split(), list.append(), tuple.index(), .....</code>                                                                                                                   |
| 모듈 | 모듈       | Python 명령어로 이루어진 Python 프로그램 파일(확장자 .py)<br><br><code>import user_python_file</code><br><br><code>sum = user_python_file.user_fn(10,6)</code><br><br><code>[user_python_fille.py<br/>def user_fn(x, y):<br/>    return x + y<br/>]</code> |
|    | 내장모듈     | Python 언어 자체에 통합되어 제공되는, C 언어로 작성된 모듈<br><br><code>import os</code><br><br><code>os.mkdir("directory")</code>                                                                                                                             |

# Function

| 내장 함수                                                                                                                                                                |                                                                                                                                                                                |                                                                                                                     |                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>A</b><br><a href="#">abs()</a><br><a href="#">aiter()</a><br><a href="#">all()</a><br><a href="#">anext()</a><br><a href="#">any()</a><br><a href="#">ascii()</a> | <b>E</b><br><a href="#">enumerate()</a><br><a href="#">eval()</a><br><a href="#">exec()</a>                                                                                    | <b>L</b><br><a href="#">len()</a><br><a href="#">list()</a><br><a href="#">locals()</a>                             | <b>R</b><br><a href="#">range()</a><br><a href="#">repr()</a><br><a href="#">reversed()</a><br><a href="#">round()</a>                                                                                                               |
| <b>B</b><br><a href="#">bin()</a><br><a href="#">bool()</a><br><a href="#">breakpoint()</a><br><a href="#">bytearray()</a><br><a href="#">bytes()</a>                | <b>F</b><br><a href="#">filter()</a><br><a href="#">float()</a><br><a href="#">format()</a><br><a href="#">frozenset()</a>                                                     | <b>M</b><br><a href="#">map()</a><br><a href="#">max()</a><br><a href="#">memoryview()</a><br><a href="#">min()</a> | <b>S</b><br><a href="#">set()</a><br><a href="#">setattr()</a><br><a href="#">slice()</a><br><a href="#">sorted()</a><br><a href="#">staticmethod()</a><br><a href="#">str()</a><br><a href="#">sum()</a><br><a href="#">super()</a> |
| <b>C</b><br><a href="#">callable()</a><br><a href="#">chr()</a><br><a href="#">classmethod()</a><br><a href="#">compile()</a><br><a href="#">complex()</a>           | <b>G</b><br><a href="#">getattr()</a><br><a href="#">globals()</a>                                                                                                             | <b>N</b><br><a href="#">next()</a>                                                                                  | <b>T</b><br><a href="#">tuple()</a><br><a href="#">type()</a>                                                                                                                                                                        |
| <b>D</b><br><a href="#">delattr()</a><br><a href="#">dict()</a><br><a href="#">dir()</a><br><a href="#">divmod()</a>                                                 | <b>H</b><br><a href="#">hasattr()</a><br><a href="#">hash()</a><br><a href="#">help()</a><br><a href="#">hex()</a>                                                             | <b>O</b><br><a href="#">object()</a><br><a href="#">oct()</a><br><a href="#">open()</a><br><a href="#">ord()</a>    | <b>V</b><br><a href="#">vars()</a>                                                                                                                                                                                                   |
|                                                                                                                                                                      | <b>I</b><br><a href="#">id()</a><br><a href="#">input()</a><br><a href="#">int()</a><br><a href="#">isinstance()</a><br><a href="#">issubclass()</a><br><a href="#">iter()</a> | <b>P</b><br><a href="#">pow()</a><br><a href="#">print()</a><br><a href="#">property()</a>                          | <b>Z</b><br><a href="#">zip()</a>                                                                                                                                                                                                    |
|                                                                                                                                                                      |                                                                                                                                                                                |                                                                                                                     | <b>-</b><br><a href="#">__import__()</a>                                                                                                                                                                                             |

## 사용자 정의 함수





여러 개의 값을 반환할 수 있음

```
def find_min_max(A) :                                # 최댓값과 최솟값을 동시에 찾아 반환
    min = A[0]
    max = A[0]
    for i in range(1, len(A)) :                      # i : 1 ~ len(A)-1
        if max < A[i] : max = A[i]                  # 최댓값 갱신
        if min > A[i] : min = A[i]                  # 최솟값 갱신
    return min, max                                   # 최솟값과 최댓값을 반환

data = [ 5, 3, 8, 4, 9, 1, 6, 2, 7 ]
x, y = find_min_max(data)                            # 최솟값과 최댓값을 반환받음
print("(min,max) = ", (x,y))                         # x와 y를 튜플로 만들어 출력
```



## 디폴트 인수

```
def sum_range(begin, end, step=1) :           # 매개변수 step이 기본 값을 가짐
    sum = 0
    for n in range(begin, end, step) :
        sum += n
    return sum
```

```
print("sum = ", sum_range(1, 10))           # step은 디폴트 값(1)으로 처리됨
```

```
print("sum = ", sum_range(1, 10, 2))        # 정상 호출. step은 2
```

## 키워드 인수

```
print("sum = ", sum_range(step=3, begin=1, end=10)) # 키워드 인수 사용
```

```
print("game ", end=" ")                     # 라인피드가 발생하지 않음(키워드 인수 사용)
```

# Global variable

```
def calc_perimeter(radius) :
    # global perimeter
    print("파이 값: ", pi)
    perimeter = 2*pi * radius
```

# 파이값 출력 --> OK

# 원 둘레 계산 --> 이상?

```
pi = 3.14159
perimeter = 0
calc_perimeter(10)
print("원둘레(r=10) = ", perimeter)
```

# 전역변수 pi : 파이값의 근사치

# 전역변수 perimeter : 원 둘레

| Command Prompt              | Output                                |
|-----------------------------|---------------------------------------|
| C:\WINDOWS\system32\cmd.exe | 파이 값: 3.14159<br>원 둘레(r=10) = 0       |
| C:\WINDOWS\system32\cmd.exe | 파이 값: 3.14159<br>원 둘레(r=10) = 62.8318 |

이상한 결과?

원하는 결과

# Module & Namespace

```
# 파일명: min_max.py
def find_min_max(A) :
    ...
    return min, max
```

# 다음 함수가 min\_max.py에 저장되어 있음  
# 최댓값과 최솟값을 동시에 찾아 반환  
# 2.7장의 find\_min\_max()함수 코드와 동일  
# 최솟값과 최댓값을 반환

```
# 파일명: sum.py
def sum_range(begin, end, step=1) :
    ...
    return sum
```

# 다음 함수가 sum.py에 저장되어 있음  
# 매개변수 step이 기본 값을 가짐  
# 2.7장의 sum\_range()함수 코드와 동일

```
# 파일명: my_job.py
import min_max          # min_max.py 모듈을 사용함
import sum               # sum.py 모듈을 사용함

data = [ 5, 3, 8, 4, 9, 1, 6, 2, 7 ]
print("(min,max) = ", min_max.find_min_max(data))
print("sum = ", sum.sum_range(1, 10))
```

# min\_max 모듈의 함수 사용  
# sum 모듈의 함수 사용

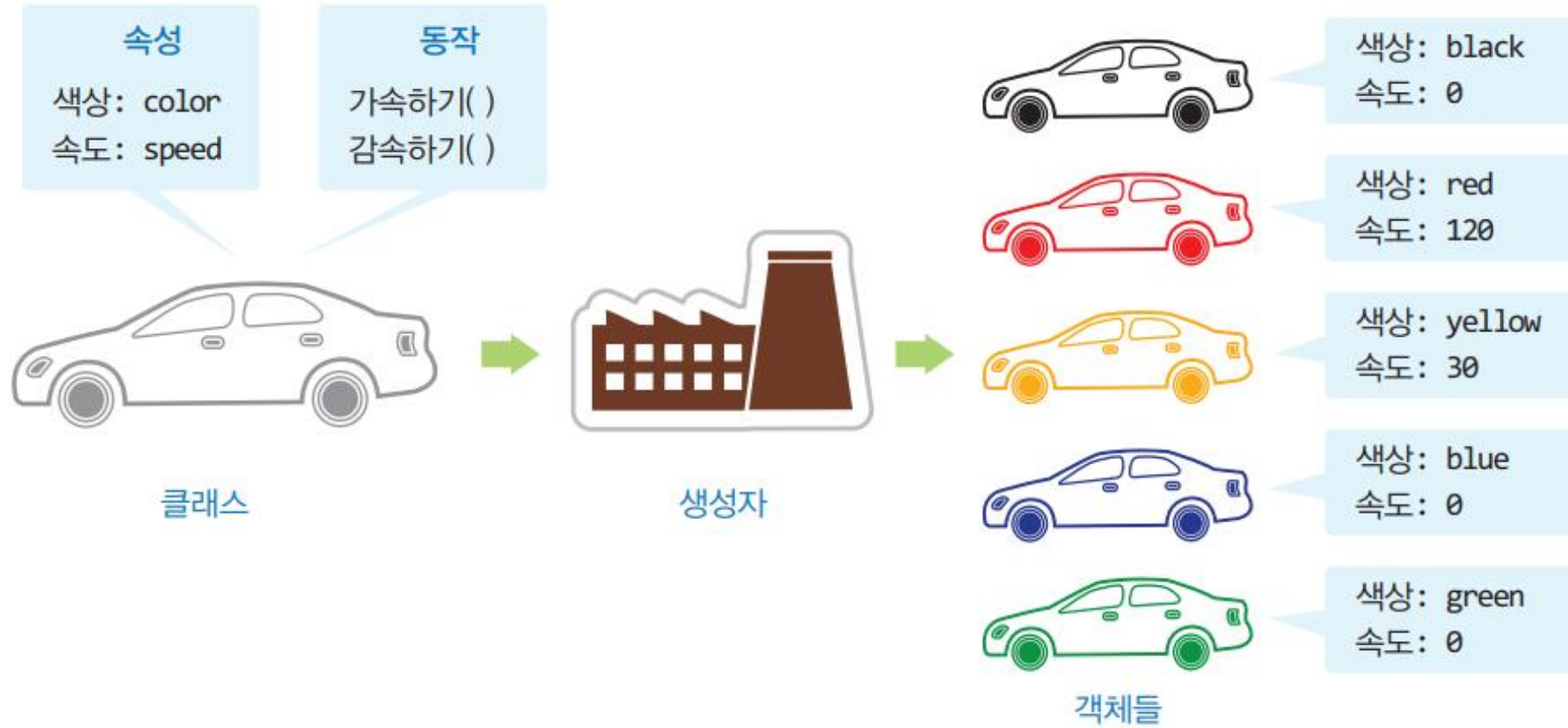
# Module & Namespace

```
from math import pow, sqrt      # math 모듈에서 pow, sqrt 식별자를 사용
result = pow(2,10)              # pow 식별자를 바로 사용 가능
dist = sqrt(1000)               # sqrt 식별자를 바로 사용 가능
```

```
# 파일명: my_job.py
from min_max import *          # min_max 모듈의 모든 식별자 사용 가능
from sum import *              # sum 모듈의 모든 식별자 사용 가능

data = [ 5, 3, 8, 4, 9, 1, 6, 2, 7 ]
print("(min,max) = ", find_min_max(data))    # 바로 사용
print("sum = ", sum_range(1, 10))            # 바로 사용
```

# Class



```
class Car :
```

# 자동차와 관련된 클래스 정의 블록이 이어짐. 들여쓰기에 유의할 것

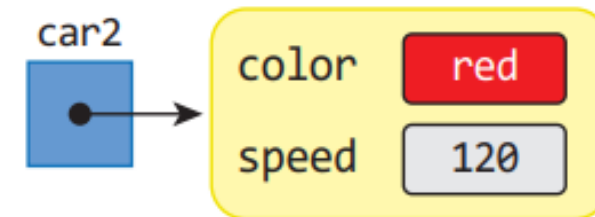
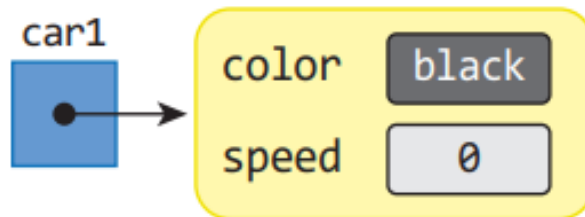
# Constructor

```

class Car :
    def __init__(self, color, speed = 0) :      # 생성자 함수
        self.color = color                    # 데이터 멤버 color 정의 및 초기화
        self.speed = speed                    # 데이터 멤버 speed 정의 및 초기화
    
```

```

car1 = Car('black', 0)      # 검정색, 속도 0
car2 = Car('red', 120)     # 빨간색, 속도 120
car3 = Car('yellow', 30)   # 노란색, 속도 30
car4 = Car('blue', 0)      # 파랑색, 속도 0
car5 = Car('green')        # 초록색, 속도 0 (디폴트 인수 사용)
    
```



# Member function

```
class Car :
```

```
# ...
```

```
def speedUp(self) : self.speed += 10
```

# 가속 동작: 속도 10 증가

```
def speedDown(self) : self.speed -= 10
```

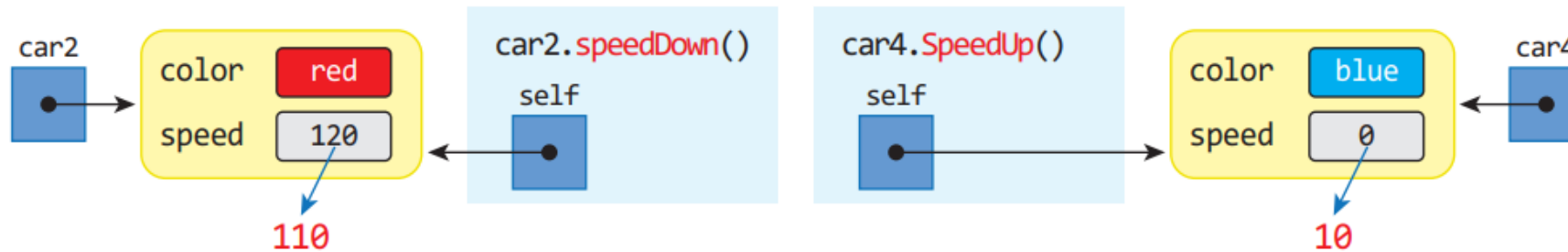
# 감속 동작: 속도 10 감소

```
car2.speedDown()
```

# car2 감속: 속도 10 감소

```
car4.speedUp()
```

# car4 가속: 속도 10 증가



```
car3.color = 'purple'
```

# car3의 색상을 purple로 변경

```
car5.speed = 100
```

# car5의 속도를 100으로 변경



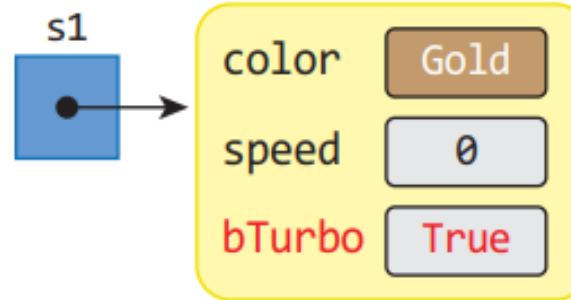
# Inheritance

```

class SuperCar(Car) :
    def __init__(self, color, speed = 0, bTurbo = True) :
        super().__init__(color, speed)      # 부모(Car)클래스의 생성자 호출
        self.bTurbo = bTurbo                # 터보모드를 위한 변수 생성 및 초기화
    
```

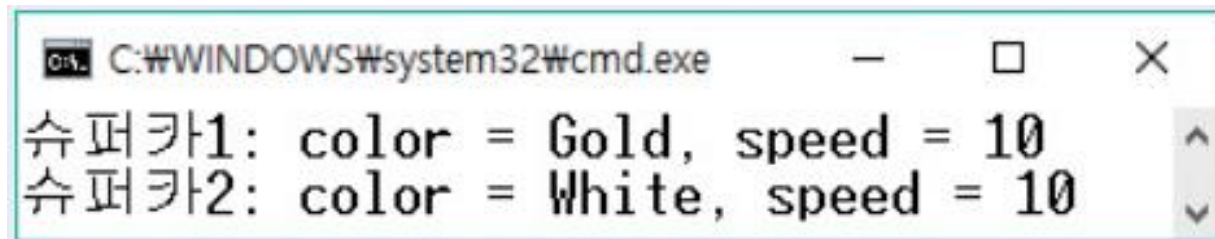
```

s1 = SuperCar("Gold", 0, True)
s2 = SuperCar("White", 0, False)
    
```



```

s1.speedUp()
s2.speedUp()
print("슈퍼카1:", s1)
print("슈퍼카2:", s2)
    
```



A screenshot of a Windows command prompt window titled 'C:\WINDOWS\system32\cmd.exe'. The output shows the results of the Python code execution:

```

슈퍼카1: color = Gold, speed = 10
슈퍼카2: color = White, speed = 10
    
```



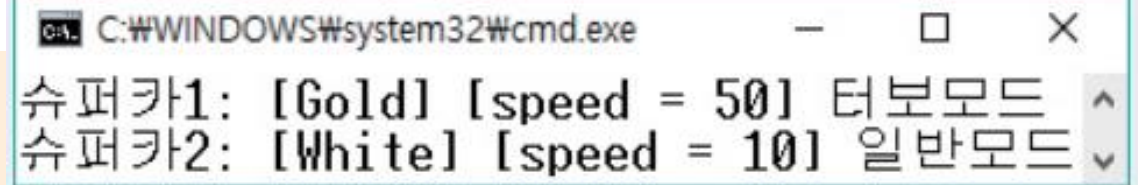
# Overriding

```
def setTurbo(self, bTurbo = True) :    # 터보모드를 On/Off하는 메소드
    self.bTurbo = bTurbo
```

```
def speedUp(self) :                    # SuperCar의 SpeedUp. 메소드 재정의
    if self.bTurbo :                  # 터보 모드이면
        self.speed += 50              # 속도가 급속히(50) 증가됨
    else :                             # 터보 모드가 아니면
        super().speedUp()             # 일반 자동차의 가속 메소드 호출
```

```
s1.speedUp()
s2.speedUp()
print("슈퍼카1:", s1)
print("슈퍼카2:", s2)
```

```
def __str__(self) :
    if self.bTurbo :
        return "[%s] [speed = %d] 터보모드" % (self.color, self.speed)
    else :
        return "[%s] [speed = %d] 일반모드" % (self.color, self.speed)
```



```
C:\WINDOWS\system32\cmd.exe
슈퍼카1: [Gold] [speed = 50] 터보모드
슈퍼카2: [White] [speed = 10] 일반모드
```

# Reference



- IT CookBook, 쉽게 배우는 자료구조 with 파이썬, 문병로, 한빛 아카데미
- 파이썬으로 쉽게 배우는 자료구조, 최영규, 천인국, 생능출판사
- <https://wikidocs.net/book/1>
- <https://wikidocs.net/84388>
- <https://www.tiobe.com/tiobe-index/>
- <https://blog.naver.com/youndok/222032150902>
- <https://python.plainenglish.io/python-lists-vs-tuples-vs-sets-vs-dictionaries-d273573dca24>
- <https://pythonflood.com/python-operator-precedence-simplifying-complex-expressions-22eb46b334>
- <https://www.anaconda.com/>
- <https://www.javatpoint.com/compiler-vs-interpreter>